

CSCI2750 Data Mining & Information Processing

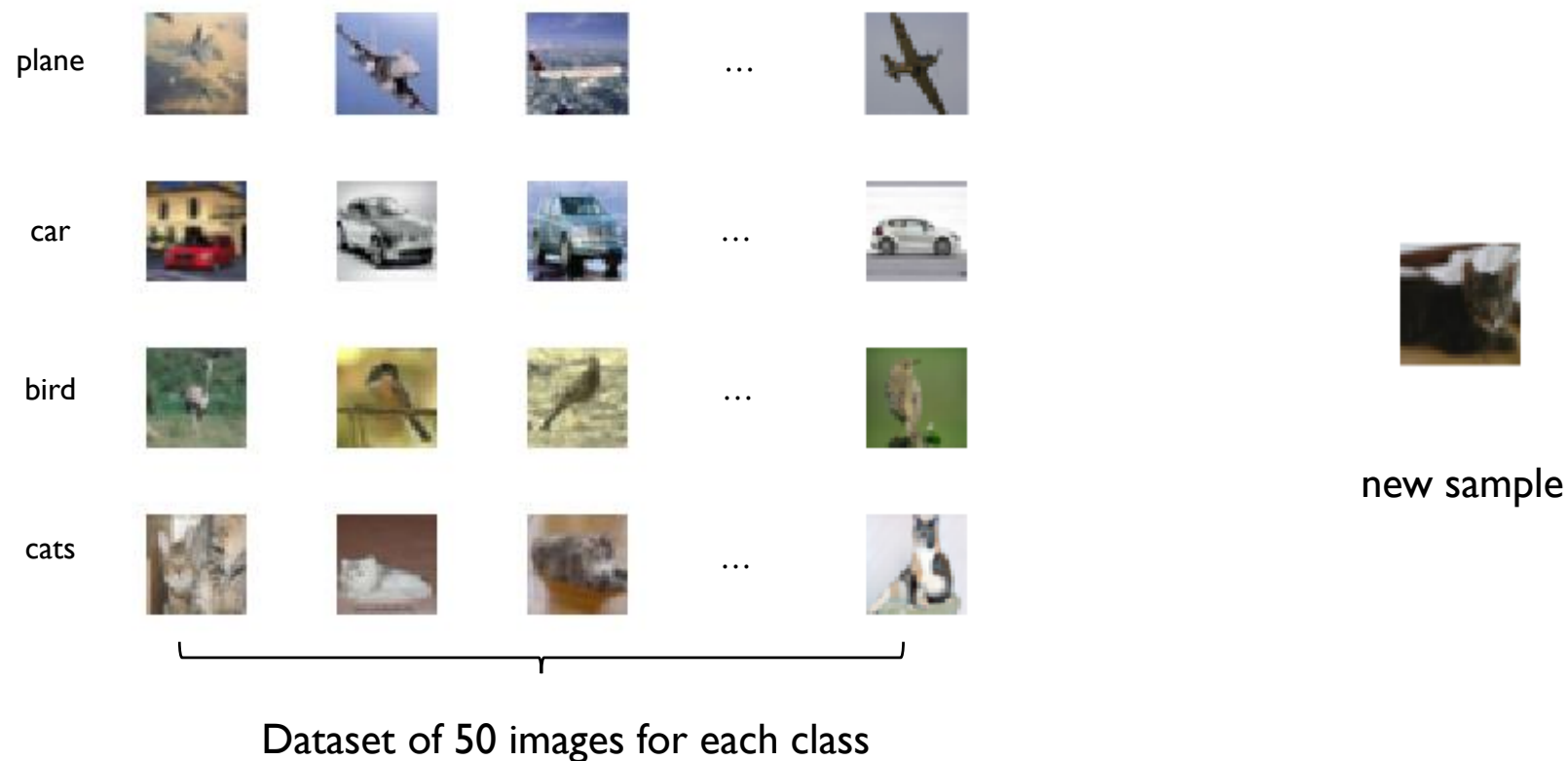
Lecture 6: Classification and The Perceptron

Wai-Yiu Keung

Term 2, 2025-26

Department of Computer Science and Engineering
The Chinese University of Hong Kong

Motivation



- suppose that we have 50 images of airplanes, 50 images of cars, 50 images of birds and 50 images of cats
- we learn how do cars, planes, birds and cats look like from these data
- given new sample on the RHS, we want to identify it's category

Supervised Learning

- general supervised learning case:
 - input: i) features of observations and ii) the corresponding labels
 - output: a model that accurately predicts the labels of unseen samples
- most useful when:
 - you don't know how to write a program to do a prediction task
 - but you have many/sufficient input-output example-pairs
- exam-practice analogy:
 - input: i) past paper/exercise/homework problems and ii) the marks/correctness of your practicing attempt
 - output: an exam-oriented strategy on how to answer questions
 - (the model is to be executed in the actual examination)

Classification

- recall that regression aims at finding a fitting $h : \mathbb{R}^n \mapsto \mathbb{R}$ for some given **numerical data**, i.e., both input and output variables are real-valued
- in many applications, however, data takes only *specific* values (e.g. categorical attributes) and thus the alphabet for both x and y are **finite discrete sets**
- example: in major subject recommendation, learning targets are

$$y \in \mathcal{Y} = \{\text{Arts, Business, Engineering, Science, Social Sci.}\}$$

and the input feature x maybe

$$x = \begin{bmatrix} \text{who is your favourite author?} \\ \text{frequency of visiting Google scholar} \\ \text{do you have a calculator with you?} \\ \vdots \\ \text{how often do you code?} \end{bmatrix};$$

we assume $x \in \mathbb{R}^n$ in this lecture

Binary Classification

- we focus on binary classification
- basic setup:
 - $\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$ are given as dataset
 - $\mathbf{x} \in \mathbb{R}^n$ is n -dimensional feature vector
 - $y \in \{+1, -1\}$ is a true/false label

this is mostly the same as regression, with a difference in the domain of y

- **task**: find $h : \mathbb{R}^n \mapsto \{-1, +1\}$ that minimizes

$$y^{(i)} - h(\mathbf{x}^{(i)}), \quad i = 1, \dots, m,$$

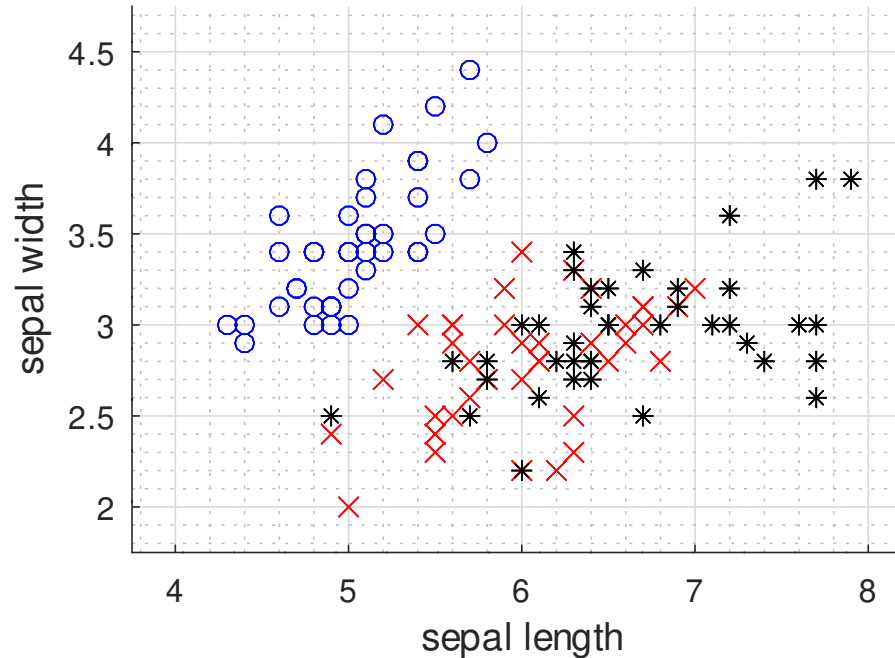
where $\hat{y}_i = h(\mathbf{x}^{(i)}) \in \{-1, +1\}$

- the function h here is called a **classifier**

Some Famous Applications of Binary Classification

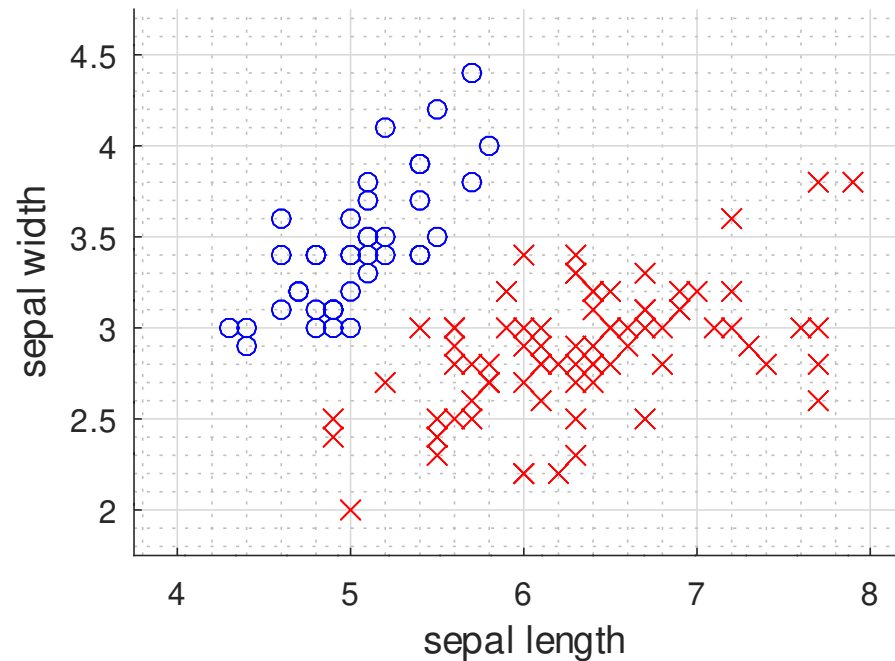
- **spam email detection:**
 - x as a feature vector that represents a certain email;
 - attributes may contain word counts of the email, number of exclamation marks and the number of ALL CAPITAL LETTER words;
 - the label y will be a spam indicator:
$$y = \begin{cases} +1 & \text{if the email is spam} \\ -1 & \text{if the email is not spam} \end{cases}$$
- **document classification:** x is a feature vector of a document; y is an indicator that whether the document belongs to a certain topic area, e.g. $y = 1$ if the document is about economy; and $y = -1$ if it isn't.
- **transaction fraud detection:** x is feature vector associated with a certain credit card holder — monthly spending, price of purchases, shops visited most etc — and $y = 1$ if the transaction is a fraudulent one; $y = -1$ otherwise

Example: Iris Dataset



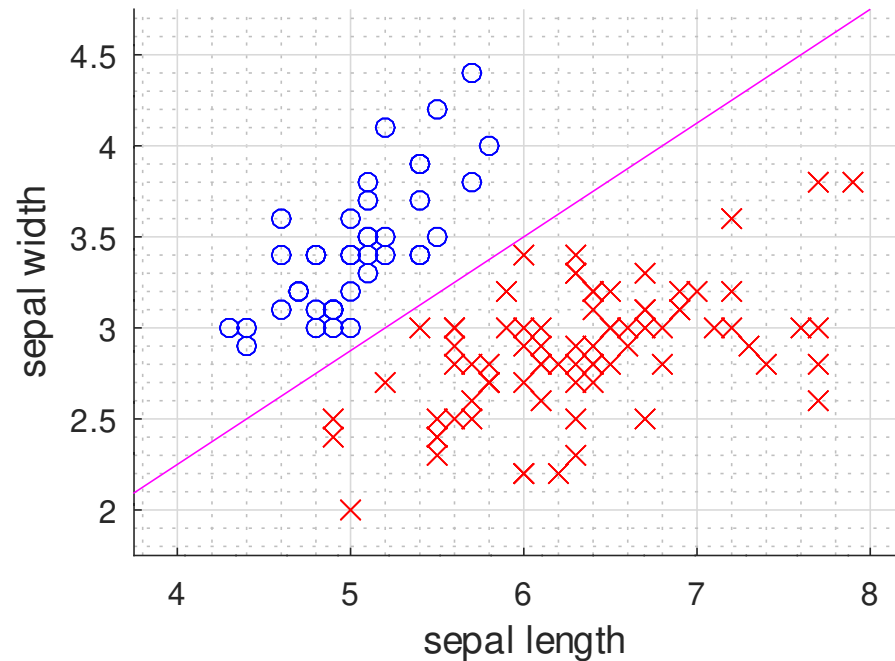
- goal: classify iris flowers using their sepal length and width
- blue circles $\circ$ are setosa; red crosses $\times$ are versicolour; black stars $*$ are virginica
- this implies the original label set is $\mathcal{Y} = \{\text{setosa}, \text{versicolour}, \text{virginica}\}$;
- next, we will turn the classification problem into a binary one

Geometric Example: Iris Dataset



- we first decide whether is it setosa
- blue circles $\circ$ are setosa; red crosses $\times$ are NOT setosa;
- the label set is now turned to $\mathcal{Y} = \{\text{setosa}, \text{not-setosa}\}$; which we can encode it into $\mathcal{Y} = \{+1, -1\}$

Geometric Example: Iris Dataset



- **linear classification**: find the line that decides whether a data point $\mathbf{x}$ has $y = 1$ or $y = -1$; the line is defined by a vector $\mathbf{w}$
- specifically, a linear classifier admits the form

$$h(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w}^T \mathbf{g} \geq 0 \\ -1 & \text{else if } \mathbf{w}^T \mathbf{g} < 0 \end{cases}$$

where $\mathbf{g} = (g_1, \dots, g_p)$ and $g_i(\mathbf{x})$ is a real-valued function defined on $\mathbf{x}$

We will introduce a simple yet powerful algorithm that solves the binary classification problem — [the perceptron](#). Apart from learning how to operate so, we will also shed some light on the theoretical guarantees it provides.

Without loss of generality, we will assume the data $\mathbf{x}^{(i)}$'s follow a distribution of zero-mean. Or, we assume standardization

$$\mathbf{x}^{(i)} \leftarrow (\mathbf{x}^{(i)} - \boldsymbol{\mu}) \oslash \boldsymbol{\sigma}$$

where $\boldsymbol{\mu} = \sum_{i=1}^m \mathbf{x}^{(i)}$, $\boldsymbol{\sigma}$ is the standard deviation of the attributes in $\mathbf{x}$, and $\oslash$ means an element-by-element division (assuming $\boldsymbol{\sigma} > 0$).

Perceptron

Let us focus on a simple case where $\mathbf{g} = \mathbf{x} - \mu(\mathbf{x}) \in \mathbb{R}^n$ is the de-meaned data points (we will extend it later); the **perceptron algorithm** is extremely easy to follow:

Algorithm: Perceptron.

- initialise $\mathbf{w}^{(1)} = \mathbf{0}$ and iterate for $t = 1, 2, \dots, T$
- in each iteration, it searches for a **violation point** $\mathbf{p}$ amongst all $\mathbf{g}^{(i)}$:
 - $\mathbf{p}$ has label $y = +1$ but $\mathbf{w}^T \mathbf{p} < 0$
 - $\mathbf{p}$ has label $y = -1$ but $\mathbf{w}^T \mathbf{p} \geq 0$
- If $\mathbf{p}$ exists, **update** $\mathbf{w}^{(t)}$ according to the following rules:
 - If $\mathbf{p}$ is associated with $y = +1$, then $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \mathbf{p}$
 - If $\mathbf{p}$ is associated with $y = -1$, then $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \mathbf{p}$

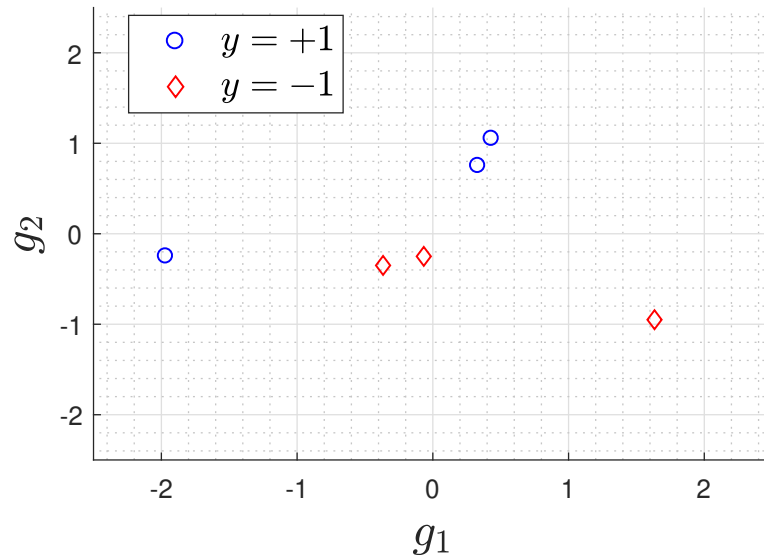
The algorithm terminates when there is no more $\mathbf{p}$ found.

Example: Perceptron Algorithm

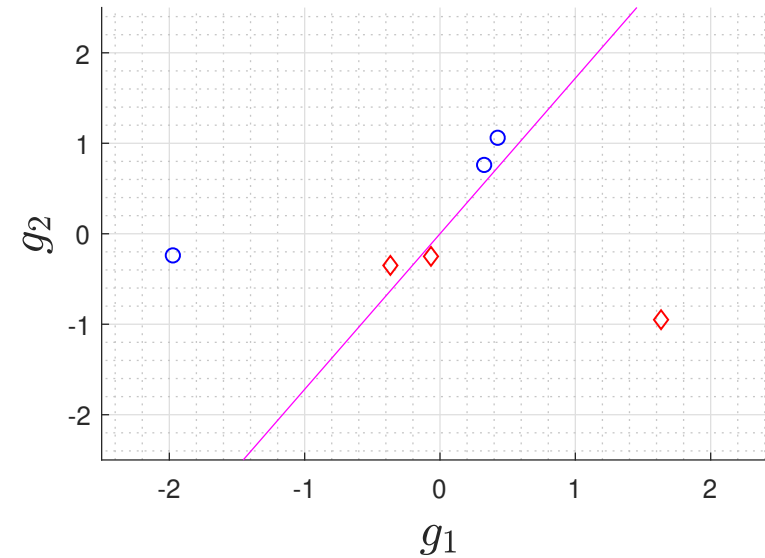
raw data $\mathbf{x}$	feature values $\mathbf{g}$	label y
$\mathbf{x}^{(1)} = (4.2, 3.4)$	$\mathbf{g}^{(1)} = (-1.97, -0.25)$	$y_1 = +1$
$\mathbf{x}^{(2)} = (6.5, 4.4)$	$\mathbf{g}^{(2)} = (0.33, 0.75)$	$y_2 = +1$
$\mathbf{x}^{(3)} = (7.8, 2.7)$	$\mathbf{g}^{(3)} = (1.63, -0.95)$	$y_3 = -1$
$\mathbf{x}^{(4)} = (6.1, 3.4)$	$\mathbf{g}^{(4)} = (-0.07, -0.25)$	$y_4 = -1$
$\mathbf{x}^{(5)} = (6.6, 4.7)$	$\mathbf{g}^{(5)} = (0.43, 1.05)$	$y_5 = +1$
$\mathbf{x}^{(6)} = (5.8, 3.3)$	$\mathbf{g}^{(6)} = (-0.37, -0.35)$	$y_6 = -1$

- initialise $\mathbf{w}^{(1)} = (0, 0)$
- iteration $t = 1$: first we see $\mathbf{w}^T \mathbf{g}^{(3)} = 0 \geq 0$ while having $y_3 = -1$; thus $\mathbf{p} = \mathbf{g}^{(3)} = (1.63, -0.95)$; we then update $\mathbf{w}^{(2)} = \mathbf{w}^{(1)} - \mathbf{p} = (1.63, -0.95)$
- iteration $t = 2$: we now see $(\mathbf{w}^{(2)})^T \mathbf{g}^{(6)} \geq 0$ but $y_1 = -1$; thus $\mathbf{p} = \mathbf{g}^{(6)} = (-0.37, -0.35)$; and we update $\mathbf{w}^{(3)} = \mathbf{w}^{(2)} + \mathbf{p} = (-1.27, 1.3)$
- iteration $t = 3$: again we see $(\mathbf{w}^{(3)})^T \mathbf{g}^{(6)} \geq 0$; thus $\mathbf{p} = (1.63, -0.95)$ and we update the $\mathbf{w}^{(4)} = \mathbf{w}^{(3)} + \mathbf{p} = (-0.9, 1.65)$
- we finally see $\text{sign}(\mathbf{w}^T \mathbf{g}^{(i)}) = y^{(i)}$ for all $i = 1, \dots, 6$ and thus the algo. terminates here

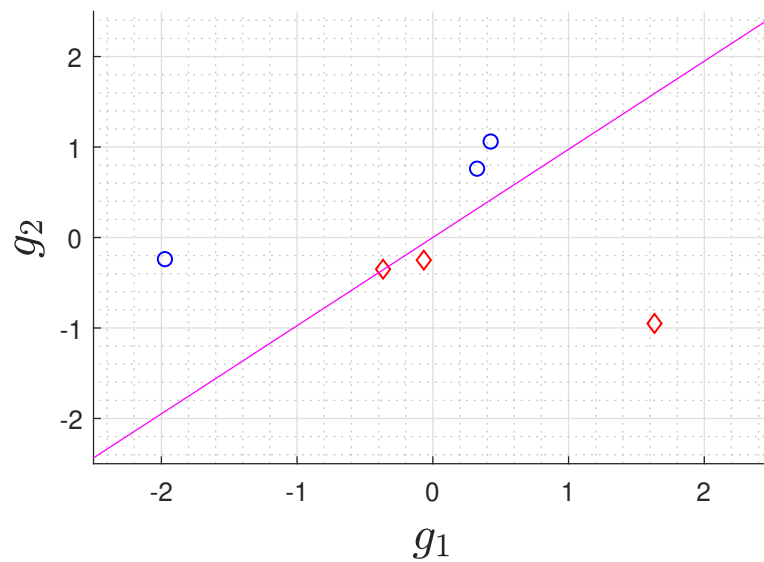
De-meaned dataset **G**



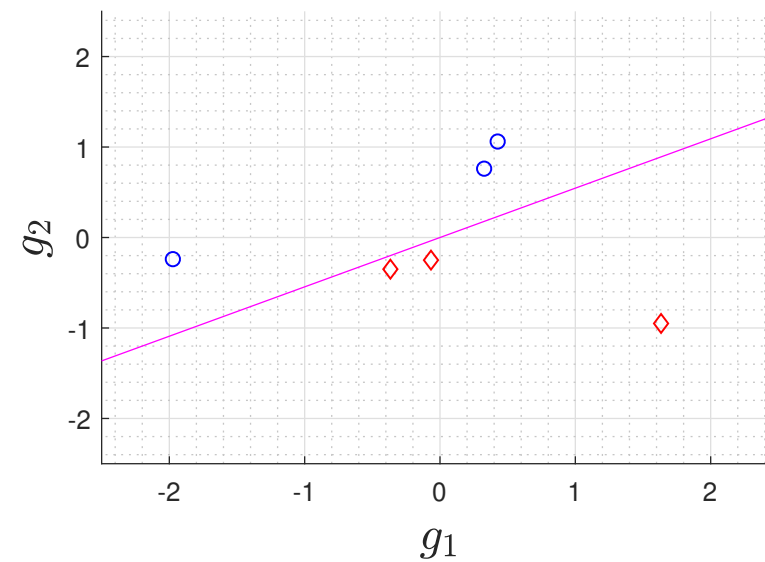
First Iteration



Second Iteration

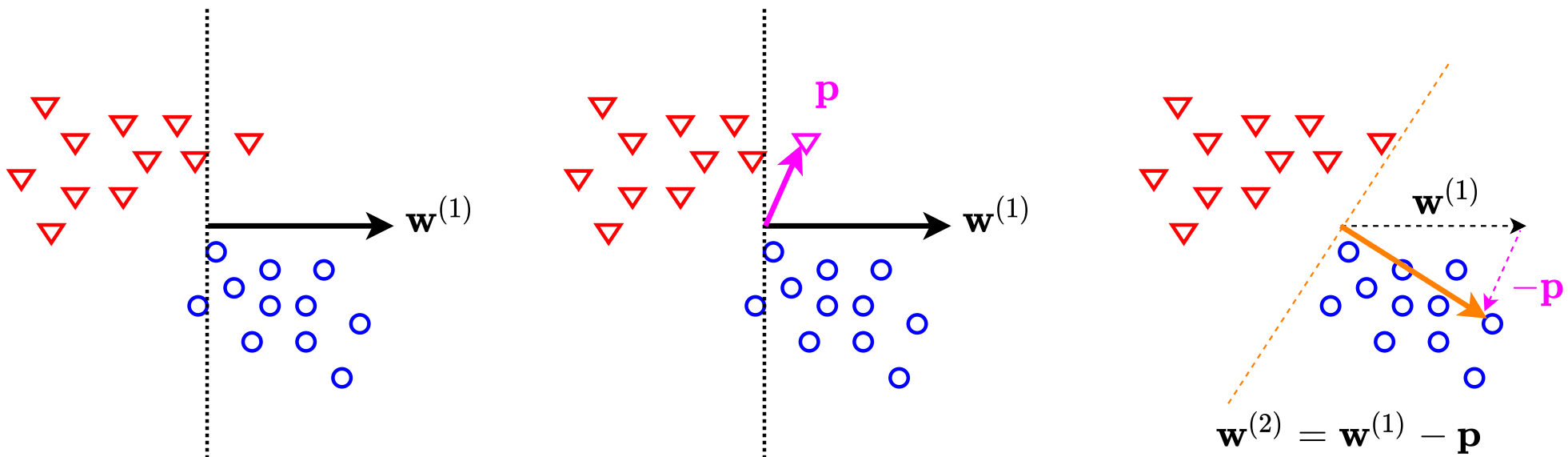


Third Iteration



The perceptron algorithm returns a line $\mathbf{w}^T \mathbf{x} = 0 \iff x_2 = \frac{-w_1}{w_2} \cdot x_1$

Perceptron: Geometric Intuition



- 2D data: blue circles are $y = 1$ and red triangles are with $y = -1$
- left: perceptron starts with $\mathbf{w}^{(1)} = (0, 0)$, corresponding to a vertical line
- middle: a violation point $\mathbf{p}$ is found and thus we update $\mathbf{w}$
- right: updated $\mathbf{w}^{(2)}$ gives a line that separates the two classes perfectly

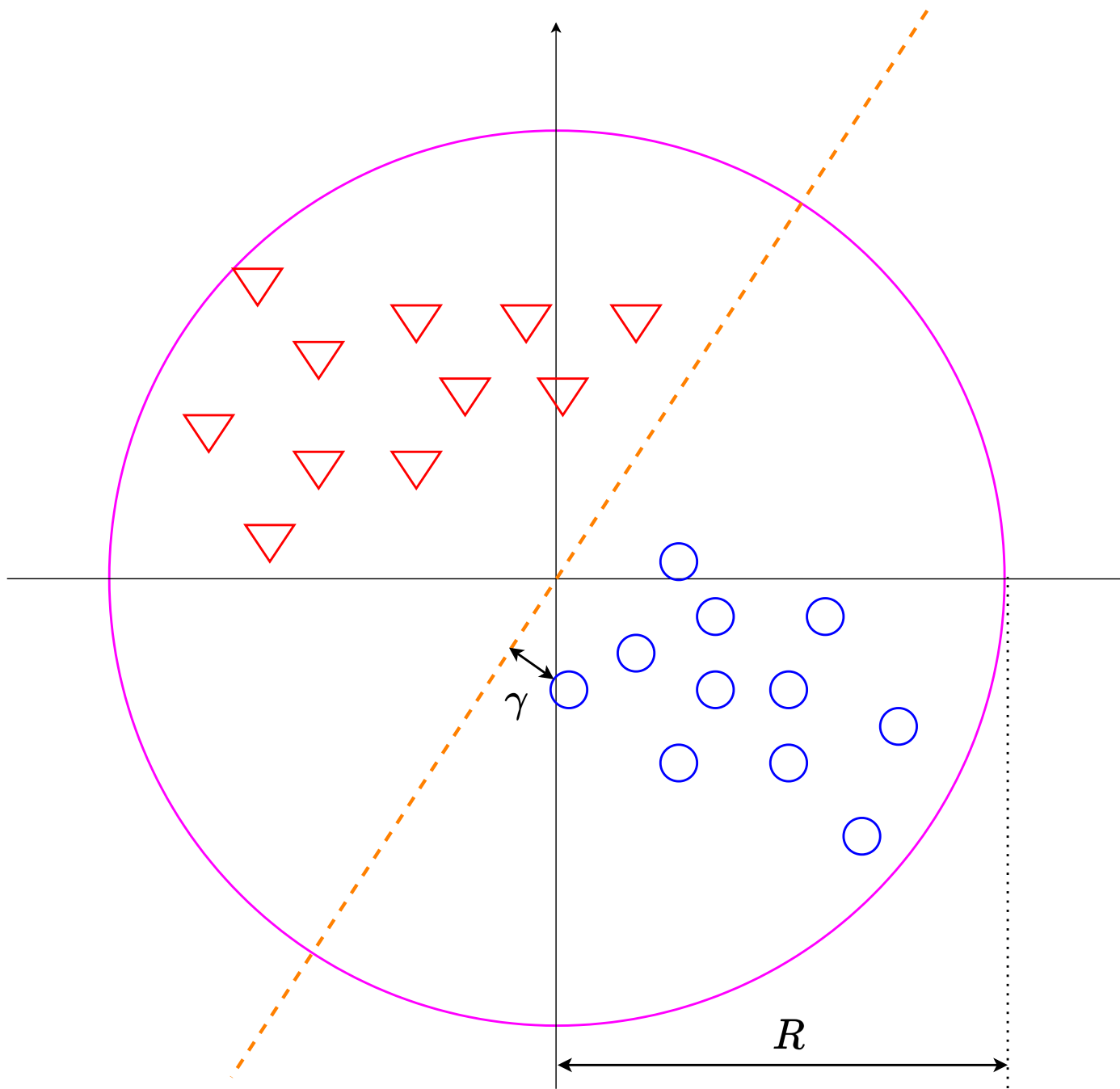
Convergence of the Perceptron

- we will now study the convergence time of the perceptron algo.
- **assumption:** the given dataset $(\mathbf{x}^{(i)}, y^{(i)})$'s are linearly separable, i.e. there **must** be at least one $\mathbf{w}^*$ such that $\text{sign}((\mathbf{g}^{(i)})^T \mathbf{w}^*) = y^{(i)}$
- **assumption:** let $\mathbf{g} = \mathbf{x}$ for simplicity; this has no impact on the proof of convergence of the algorithm
- **radius:** define $R = \max_{i=1, \dots, m} \|\mathbf{x}^{(i)}\|_2$; in other words all data points are lying inside the circle (or ball) with radius R
- **margin:** denoted by $\mathbf{u} = \mathbf{w} / \|\mathbf{w}\|_2$ the **unit vector** of the separation vector $\mathbf{w}^*$; define:

$$\gamma = \max_{\mathbf{w}} \min_{i=1, \dots, m} |\mathbf{u}^T \mathbf{x}^{(i)}| = \min_{i=1, \dots, m} \left| \frac{(\mathbf{w}^*)^T \mathbf{x}^{(i)}}{\|\mathbf{w}^*\|_2} \right|$$

i.e. margin is the distance between the separation plane $\mathbf{w}^*$ and the closest data point

- see next slide for a 2D illustration of R and γ given some data points



Theorem. The perceptron algorithm terminates after at most $(R/\gamma)^2$ updates of $\mathbf{w}^{(t)}$, assuming that the input dataset is linearly separable.

Some settings before we start proofing:

- $\mathbf{w}^{(1)} = (0, 0, \dots, 0)$
- $\mathbf{w}^{(t)}$ = the updated weight vector in the t -th iteration
- let K be the maximum number of adjustment

we will proof the theorem in a constructive manner. Specifically, we will show two inequalities:

$$(\mathbf{w}^{(K)})^T \mathbf{u}^* \geq K \cdot \gamma; \text{ and} \tag{1}$$

$$\|\mathbf{w}^{(K)}\|_2^2 \leq KR^2 \tag{2}$$

and finally combine them to arrive $K \leq R^2/\gamma^2$ as desired.

Target: $\mathbf{w}^{(K)} \mathbf{u}^* \geq K \cdot \gamma$

- consider the k -th update of the perceptron: $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} \pm \mathbf{p}$ given the **violation point** $\mathbf{p}$; we have

$$(\mathbf{w}^{(k)})^T \mathbf{u}^* = (\mathbf{w}^{(k-1)})^T \mathbf{u}^* \pm \mathbf{p}^T \mathbf{u}^*$$

note that $\mathbf{p}$ is a point $\mathbf{x}$, so according to the definition of γ we know that $\mathbf{p}^T \mathbf{u}^* \geq \gamma$ and thus we see

$$(\mathbf{w}^{(k)})^T \mathbf{u}^* \geq (\mathbf{w}^{(k-1)})^T \mathbf{u}^* + \gamma$$

for $k = 1, 2, \dots, K$

- using this argument we are able to induce

$$\begin{aligned} (\mathbf{w}^{(K)})^T \mathbf{u}^* &\geq (\mathbf{w}^{(K-1)})^T \mathbf{u}^* + \gamma \\ &\geq (\mathbf{w}^{(K-2)})^T \mathbf{u}^* + 2\gamma \\ &\vdots \\ &\geq (\mathbf{w}^{(0)})^T \mathbf{u}^* + K\gamma \\ &= K\gamma \end{aligned} \tag{1}$$

as desired

Target: $\|\mathbf{w}^{(K)}\|_2^2 \leq KR^2$

- recall the update rule $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} \pm \mathbf{p}$; using the norm of sum:

$$\begin{aligned}\|\mathbf{w}^{(k)}\|_2^2 &= \|\mathbf{w}^{(k-1)}\|_2^2 + \|\mathbf{p}\|_2^2 + 2\|\mathbf{w}^{(k-1)}\|_2 \cdot \|\mathbf{p}\|_2 \cdot \cos(\theta) \\ &\leq \|\mathbf{w}^{(k-1)}\|_2^2 + \|\mathbf{p}\|_2^2\end{aligned}$$

where θ is the angle between $\mathbf{w}^{(k-1)}$ and $\mathbf{p}$

- note that $\mathbf{p}$ is also a data point $\mathbf{x}^{(i)}$ so its norm should be no larger than R
- hence, we are able to induce

$$\begin{aligned}\|\mathbf{w}^{(K)}\|_2^2 &\leq \|\mathbf{w}^{(K-1)}\|_2^2 + R^2 \\ &\leq \|\mathbf{w}^{(K-2)}\|_2^2 + 2R^2 \\ &\vdots \\ &\leq \|\mathbf{w}^{(0)}\|_2^2 + KR^2 \\ &= KR^2\end{aligned}\tag{2}$$

as desired

Combining the two inequalities.

- consider the square of the inequality (1): $\left((\mathbf{w}^{(K)})^T \mathbf{u}^*\right)^2 \geq K^2 \gamma^2$, we can evaluate:

$$K^2 \gamma^2 = \sum_{i=1}^n |w_i^{(K)} \cdot u_i^*|^2 = \sum_{i=1}^n |w_i^{(K)}|^2 \cdot |u_i^*|^2 \leq \|\mathbf{w}^{(K)}\|_2^2 \cdot \|\mathbf{u}^*\|_2^2$$

wherein we have used the fact $\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos(\theta)$ to construct the last inequality;

- recall that $\mathbf{u} = \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$ is a **unit vector**, which means $\|\mathbf{u}\|_2 = 1$, we arrive

$$K^2 \gamma^2 \leq \|\mathbf{w}^{(K)}\|_2^2 \leq KR^2$$

note the RHS upper bound is inequality (2)

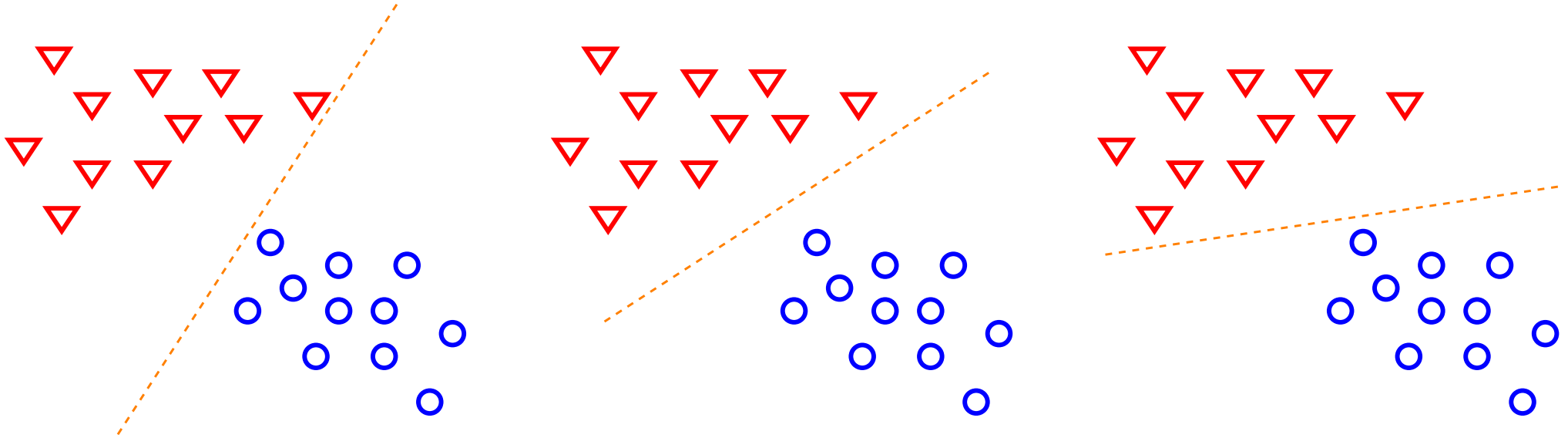
- rearranging the term allows us to arrive

$$K \leq (R/\gamma)^2$$

i.e. the theorem is proofed. □

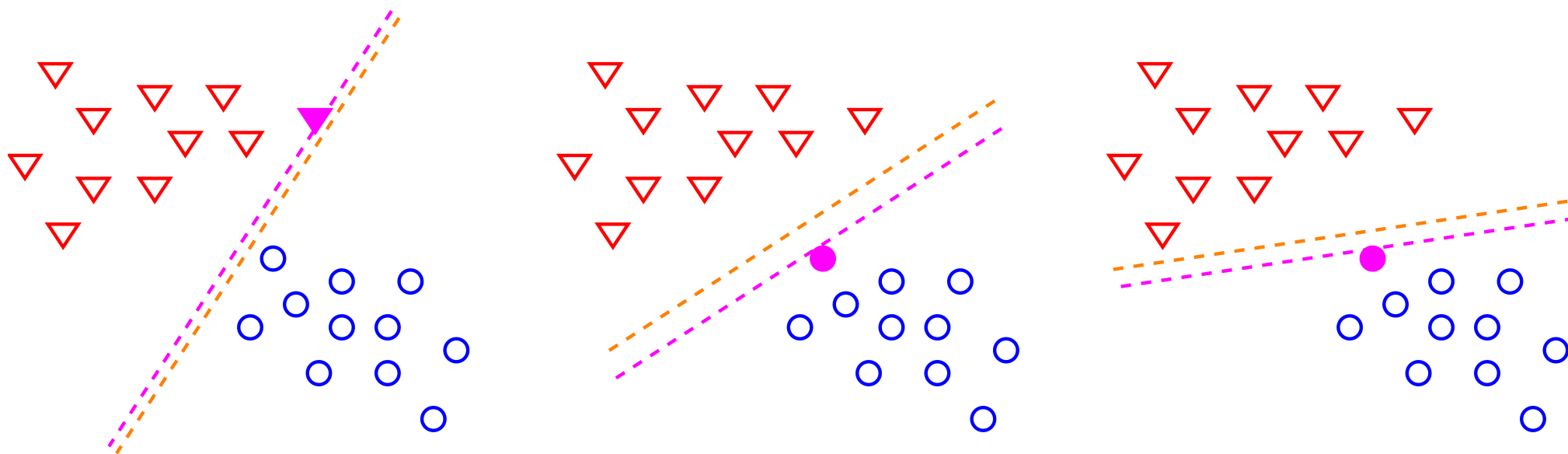
Having the perceptron algorithm introduced and the theoretical guarantee discussed, we take one step forward to check on a tiny modification of the algo in order to provide an improvement of the outcome classifier w^* .

Margin Perceptron: Motivation



- we now learnt that assuming the dataset is linearly separable the perceptron must return a separation line w in no more than $(R/\gamma)^2$ updates
- consider the above dataset — which is linearly separable — and we observe there are **more than one** possible w to separate the two classes
- note that all the above 3 lines are possible to be returned by the perceptron!
- **think:** but which split w is “better”, or “the best”?

Margin Perceptron: Motivation



- recall that the margin $\gamma = \min |\mathbf{u}^T \mathbf{x}|$ is the distance between the separation line and the nearest data point $\mathbf{x}$
- the physical meaning of margin γ is the width between the orange and pink dashed lines in the above figures
- maximum margin is believed to be the best split, i.e. the middle one

Margin Perceptron: Set-up

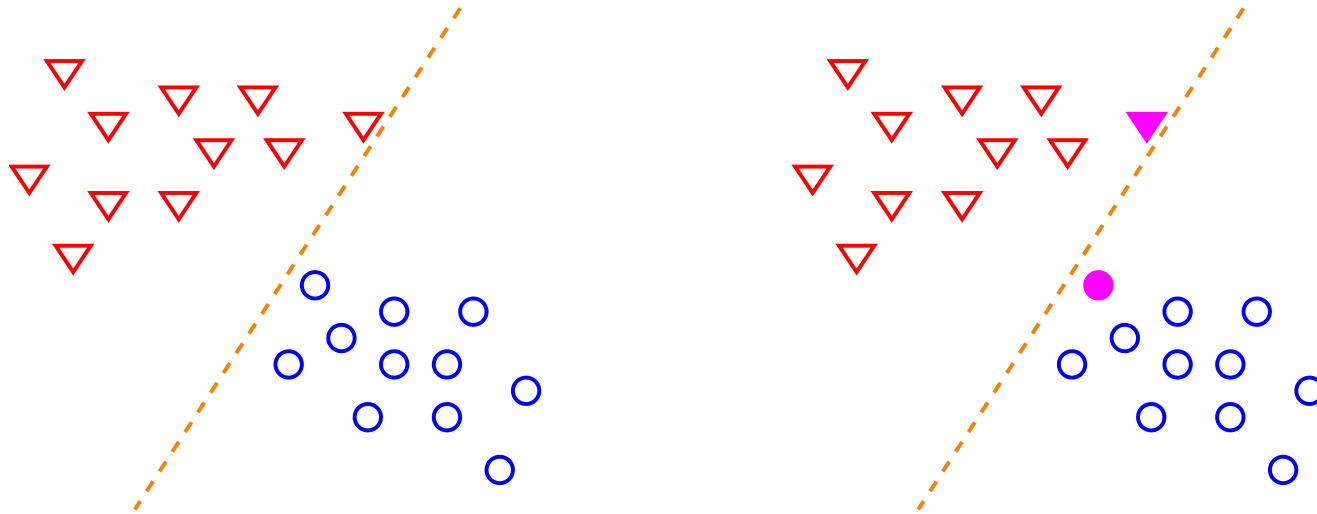
- denote $\mathcal{W}$ as the set of all possible separation vector $\mathbf{w}$ and $\gamma(\mathbf{w})$ is the margin of the respective $\mathbf{w}$; our task is to find

$$\mathbf{w}_{\text{opt}} = \max_{\mathbf{w} \in \mathcal{W}} \gamma(\mathbf{w})$$

that gives the **maximum margin**

- searching for the solution to this optimisation problem can be approached by re-formulating the problem as a convex programme (we will come back to this next lecture)
- but for this lecture we rely on the perceptron to find **an approximate solution**

Margin Perceptron: Intuition



(a)

(b)

- the idea is very straightforward: perceptron makes an update to $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y^{(i)} \cdot \mathbf{x}^{(i)}$ only if $\mathbf{x}^{(i)}$ is mis-classified by $\mathbf{w}^{(t)}$
- we modify the definition of **violation point** p as either be it mis-classified OR too close to the separation plane
- the above figure illustrates the difference between usual perceptron and the margin perceptron: (a) will accept the current split as there is no mis-classified point; but (b) will regard the **pink data points** as violation points

Margin Perceptron: Algorithm

Algorithm: Margin Perceptron.

- **Input:** data points and labels $\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$ and a small margin ε
- initialise $\mathbf{w}^{(1)} = \mathbf{0}$ and iterate for $\ell = 1, 2, \dots, L$
- in each iteration, it searches for a violation point $\mathbf{p}$ among all $\mathbf{x}^{(i)}$:
 - $\mathbf{p}$ has label $y \in \{-1, +1\}$ but $\text{sign}(\mathbf{w}^{(\ell)T} \mathbf{p}) \neq y$
 - the distance between $\mathbf{p}$ and $\mathbf{w}^{(\ell)}$ is too small, i.e. $|\mathbf{p}^T \mathbf{u}^{(\ell)}| \leq \varepsilon$
- If $\mathbf{p}$ exists, update $\mathbf{w}^{(\ell)}$ according to the following rules:

$$\mathbf{w}^{(\ell+1)} = \mathbf{w}^{(\ell)} + y \cdot \mathbf{p}$$

The algorithm terminates when there is no more $\mathbf{p}$ found.

Multi-Class Perceptron

- thus far, we've focused on binary classification $y \in \{\pm 1\}$; now we extend the perceptron to handle the multi-class case $y \in \mathcal{Y} = \{1, 2, \dots, K\}$
- setup: $\mathbf{x}^{(i)} \in \mathbb{R}^n$ is the feature vector, $y^{(i)}$ is the label, and we want to find $h : \mathbb{R}^n \mapsto \mathcal{Y}$ that minimizes $\sum_{i=1}^m |y^{(i)} - h(\mathbf{x}^{(i)})|^2$
- **generalized linear classifiers** are defined by K linear separators:

$$\{\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \dots, \mathbf{w}_K\}, \quad \mathbf{w}_k \in \mathbb{R}^n;$$

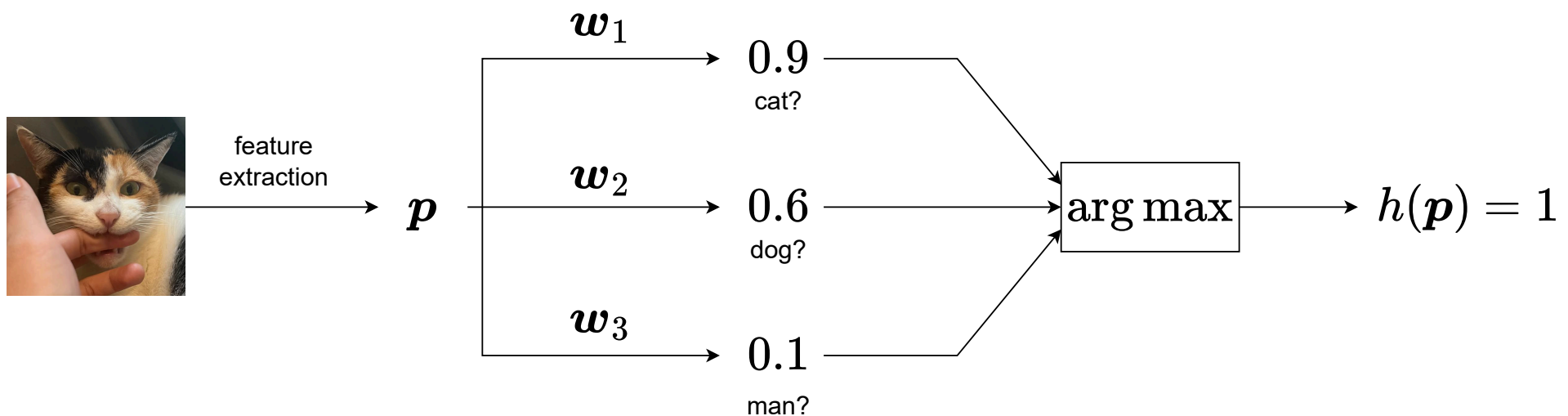
and the predicted class of a point $\mathbf{p}$ is given by

$$h(\mathbf{p}) = \arg \max_{k=1, \dots, K} \mathbf{w}_k^\top \mathbf{p}$$

in other words, it returns class label which has the largest dot product with itself

An Intuitive Illustration

- suppose you're given with a bunch of m images of cats, dogs and human; after some feature extraction you obtained the data set $\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$
- you then built exactly three binary classifiers $\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3$, with each of them being able to classify a one-versus-all problem
 - effectively, $\mathbf{w}_k$'s are “cat detector”, “dog detector” and “human detector”
 - with K classes comes K binary classifiers
- the overall multiclass linear classification process can be seen as follows:



Multi-Class Perceptron: Algorithm

Algorithm: Multiclass Perceptron.

- **Input:** data points and labels $\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^m$
- initialise $\mathbf{w}_1 = \mathbf{w}_2 = \dots = \mathbf{w}_K = \mathbf{0}$
- **while** there is a violation point $\mathbf{p}$ (or whenever $\{\mathbf{w}_k\}$ do not classify every point), **do**:
 - set ℓ = the **actual label** of $\mathbf{p}$
 - set z = the **mis-predicted label** of $\mathbf{p}$

and update the following:

$$\mathbf{w}_\ell \leftarrow \mathbf{w}_\ell + \mathbf{p}$$

$$\mathbf{w}_z \leftarrow \mathbf{w}_z - \mathbf{p}$$

s.t. $\mathbf{w}_\ell$ has bigger dot product with $\mathbf{p}$, and $\mathbf{w}_z$ will have smaller dot product with $\mathbf{p}$

Linear Separability

- we have almost always been focusing on the case $g = x$ (or $(1, x)$) for simplicity in our analysis on the perceptron; this is not true in practice — generally g is some more complicated functions of x that drives our focus away from the algo.
- the reason why we ignored the effect of $g_i(x)$'s and remain focused with x is that even it doesn't matter to the performance of the algorithm — you'll see this upon substituting every x by g
- the underlying assumption we used is that the dataset has features x being **linearly separable**, i.e., there exists $\{w_k\}$'s that correctly classify all points, and that

$$w_\ell^\top p > w_z^\top p, \quad z \neq \ell$$

for all points p with label ℓ ; we will slightly cover the non-separable case later in this course